

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("diabetes.csv")

print("First 5 rows:")
print(df.head())

print("\nShape of dataset:")
print(df.shape)

# Select numerical variable
glucose = df["Glucose"]

print("Selected variable: Glucose")
print("Mean Glucose:", glucose.mean())
print("Standard Deviation:", glucose.std())

# Number of bootstrap samples
n_bootstrap = 1000

# Store bootstrap means
bootstrap_means = []

# Bootstrap sampling
for i in range(n_bootstrap):
    sample = glucose.sample(
        n=len(glucose),
        replace=True,
        random_state=i
    )

    bootstrap_means.append(sample.mean())

bootstrap_means = np.array(bootstrap_means)

print("Number of bootstrap samples:", len(bootstrap_means))
print("First 10 bootstrap means:")
print(bootstrap_means[:10])

print("Original sample mean:", glucose.mean())
print("Bootstrap mean:", bootstrap_means.mean())
print("Bootstrap standard deviation:", bootstrap_means.std())

# 95% Bootstrap Confidence Interval

lower = np.percentile(bootstrap_means, 2.5)
upper = np.percentile(bootstrap_means, 97.5)

```

```

print("95% Bootstrap Confidence Interval:")
print("Lower limit:", lower)
print("Upper limit:", upper)

plt.figure(figsize=(8, 5))

plt.hist(bootstrap_means, bins=30, edgecolor="black")

plt.axvline(
    lower,
    linestyle="--",
    label="Lower 95% CI"
)

plt.axvline(
    upper,
    linestyle="--",
    label="Upper 95% CI"
)

plt.axvline(
    glucose.mean(),
    linestyle="-",
    label="Sample Mean"
)

plt.title("Bootstrap Distribution of Mean Glucose")
plt.xlabel("Bootstrap Mean")
plt.ylabel("Frequency")
plt.legend()

plt.show()

# Divide data into two groups

group_0 = df[df["Outcome"] == 0]
group_1 = df[df["Outcome"] == 1]

print("Non-diabetic group:")
print("Number of observations:", len(group_0))
print("Mean Glucose:", group_0["Glucose"].mean())

print("\nDiabetic group:")
print("Number of observations:", len(group_1))
print("Mean Glucose:", group_1["Glucose"].mean())

# Calculate observed difference

mean_0 = group_0["Glucose"].mean()
mean_1 = group_1["Glucose"].mean()

```

```

observed_difference = mean_1 - mean_0

print("Mean Glucose - Non-diabetic:", mean_0)
print("Mean Glucose - Diabetic:", mean_1)
print("Observed difference:", observed_difference)

# Number of permutations
n_permutations = 1000

permutation_differences = []

# Original glucose values
glucose_values = df["Glucose"].values

# Original group labels
outcomes = df["Outcome"].values.copy()

for i in range(n_permutations):

    # Randomly shuffle outcome labels
    shuffled_outcomes = np.random.permutation(outcomes)

    # Separate glucose values using shuffled labels
    shuffled_group_0 = glucose_values[shuffled_outcomes == 0]
    shuffled_group_1 = glucose_values[shuffled_outcomes == 1]

    # Calculate difference in means
    difference = (
        shuffled_group_1.mean()
        - shuffled_group_0.mean()
    )

    permutation_differences.append(difference)

permutation_differences = np.array(permutation_differences)

print("Number of permutations:", len(permutation_differences))

print("First 10 permutation differences:")
print(permutation_differences[:10])

print("Mean of permutation differences:",
      permutation_differences.mean())

print("Standard deviation:",
      permutation_differences.std())

print("Minimum difference:",
      permutation_differences.min())

```

```

print("Maximum difference:",
      permutation_differences.max())

# Calculate permutation p-value

p_value = np.mean(
    np.abs(permutation_differences)
    >= np.abs(observed_difference)
)

print("Observed difference:", observed_difference)
print("Permutation p-value:", p_value)

plt.figure(figsize=(8, 5))

plt.hist(
    permutation_differences,
    bins=30,
    edgecolor="black"
)

plt.axvline(
    observed_difference,
    linestyle="--",
    label="Observed Difference"
)

plt.axvline(
    -observed_difference,
    linestyle="--"
)

plt.title("Permutation Distribution of Difference in Mean Glucose")
plt.xlabel("Difference in Group Means")
plt.ylabel("Frequency")

plt.legend()

plt.show()

alpha = 0.05

print("\nFinal Decision:")

if p_value < alpha:
    print("Reject H0")
    print(
        "There is a statistically significant "
        "difference in mean glucose levels "
        "between diabetic and non-diabetic groups."
    )

```

```
else:  
    print("Fail to reject H0")  
    print(  
        "There is no statistically significant "  
        "difference in mean glucose levels "  
        "between diabetic and non-diabetic groups."  
    )
```